Group 10 Homework 1

Lunazzi Chiara s333949 | Pangrazi Federico s322215 | Parisini Gabriele s345139

26 November 2024

1 Exercise 1: Timeseries Processing for Memory Optimization

The following amount of memory in GB is needed in order to provide the monitoring service for 1000 clients:

$$\text{Memory occupied by the two non-aggregated timeseries (uncompressed)}$$

$$\approx \frac{2592000 \text{ s}}{2 \text{ s}} \cdot 16 \text{ bytes} \cdot 2 = 20736000 \text{ bytes} \cdot 2 = 39.55 \text{ MB}$$

$$\text{Memory occupied by the six aggregated timeseries (uncompressed)}$$

$$\approx \frac{31536000 \text{ s}}{3600 \text{ s}} \cdot 16 \text{ bytes} \cdot 6 = 140160 \text{ bytes} \cdot 6 = 0.802 \text{ MB}$$

$$\text{Memory occupied by the two non-aggregated timeseries (compressed)}$$

$$\approx 39.55 \text{ MB} - 90\% = 3.955 \text{ MB}$$

$$\text{Memory occupied by the six aggregated timeseries (compressed)}$$

$$\approx 0.802 \text{ MB} - 90\% = 0.0802 \text{ MB}$$

$$\text{Total memory dedicated to one user}$$

$$\approx 3.955 \text{ MB} + 0.0802 \text{ MB} = 4.0352 \text{ MB}$$

$$\text{Total memory dedicated to 1000 users}$$

$$\approx 4.0352 \text{ MB} \cdot 1000 \approx 3.94 \text{ GB}$$

Firstly, the number of records stored during the retention period is calculated dividing the retention period in seconds by the collection (or aggregation) interval in seconds. The amount of memory occupied by each record is 16 bytes, hence the amount of memory occupied (in bytes) by each time series is equal to the number of records multiplied by 16. The final compressed memory usage is calculated subtracting the average compression ratio of 90% (Gorilla Algorithm).

2 Exercise 2: Voice Activity Detection Optimization & Deployment

2.1 Hyperparameters analysis and effects

The VAD optimization process takes into account the impact of four different hyperparameters over accuracy and latency:

- **frame length**: The frame length defines the number of samples taken from the signal for each segment to analyze the frequency content. Each of these segments is called a frame. Longer frames can capture more frequency information at the cost of time resolution of the spectrogram, usually increasing **accuracy**. Longer frames generally increase also the **latency** but powers of 2 are an exception, decreasing the latency. This happens because the FFT algorithm uses recursion to perform the DFT computation taking into account smaller frames; for example, referring to our case, the frame length is a power of 2 and the sampling rate

is $16kHz$, so the samples in a frame are a power of 2 too, making it possible to split them in half for each recursive step. So, the computational complexity is reduced from $O(n^2)$ to $O(n\log(n))$, where n is the number of samples for each frame (for example, if the frame length is 8 ms, $n = 16000 * 0.008 = 128 = 2^7$).

- **frame step**: The frame step (or hop length) is the number of samples between the start of consecutive frames. This value determines the overlap between them; a shorter frame step can improve **accuracy** because each part of the signal is analyzed more times, reducing the chance of losing segments of the voice but it also could increase the **latency**, because the system would have to elaborate more frames.

- **dB threshold**: This parameter determines the minimum energy level of the signal (in dB) that must be exceeded for an audio segment to be considered as an entry. In terms of **accuracy**, a too low threshold value can lead to false positives, where background noises are mistakenly classified as a voice. Conversely, a too high threshold can cause false negatives, where low volume voice segments are not detected.

- **duration threshold**: This parameter specifies the minimum duration (in ms) that a signal must maintain over the dB threshold to be considered a voice signal. In terms of **accuracy** a too short duration threshold can increase false positives by detecting short noises as a voice, while too long ones can increase false negatives, ignoring short segments of real voice.

2.2 Hyperparameters search spaces and results

To look for the best hyperparameters configuration, a grid search was conducted over the following search spaces:

- **frame length (in ms)**: the space used for the frame length is D=$\{8, 16, 32, 64\}$. Note that they're all powers of 2 and chosen such that they are near the average length of an english phoneme ($[10, 50]$)

- **frame step (in ms)**: the frame step depends on the value of the frame length since it is chosen such that the overlap is in $\{0\%, 25\%, 50\%, 75\%\}$ with respect to the frame length value. Typically, a low overlap reduces the amount of data to process but implies a lower quality of the analysis, while a high overlap requires a higher computational cost but ensures a higher temporal resolution and a better quality of the analysis;

- **dB threshold (in dB)**: dB threshold is set to be in $\{5, 10, 20\}$. These values appear to be a good compromise between the microphone sensitivity and the presence of background noise. A typical value of dB threshold for voice recognition is 10 dB;

- **duration threshold (in ms)**: the duration threshold is selected from the set $\{100, 200, 300\}$. These value take into account the duration of the most common short english words.

Taking these premises into account, it is possible to evaluate the results presented in Table 1; this table summarizes the values corresponding to an accuracy higher than 97.6%. The selected hyperparameters configuration (in yellow) ensures faster processing than the defualt settings of the *vad_latency.py* script because of all the motivations discussed in the previous sections: for instance, the default configuration has got a frame length of 40 ms, which is not a power of 2, hence not optimal in terms of latency.

Frame length (ms)	Frame step (ms)	dBthres (dB)	Duration threshold (ms)	Accuracy (%)	VAD Latency (ms)
8	2	20	100	97.67	17.5 +/- 0.2
16	12	10	100	97.67	18.8 +/- 0.2
32	32	10	100	97.67	18.9 +/- 0.2
32	24	10	100	97.67	19.3 +/- 0.3
32	16	10	100	97.89	19.7 +/- 1.1
64	48	10	100	97.78	18.9 +/- 0.3

Table 1: Best performing Hyperparameters combinations